AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
MECHATRONICS ENGINEERING DEPARTMENT

Summer, 2026

CSE480: Machine Vision

Milestone 1

Build a reusable image-processing library that reconstructs a jigsaw puzzle from its shuffled pieces using classical techniques alone. The library must expose the individual operations as documented, independently testable functions, and must provide an end-to-end routine that accepts a scrambled puzzle and returns the reconstructed image together with a numerical measure of reconstruction quality. The required dataset can be accessed through the following link: Dataset. The testing puzzles may contain pieces presented in different orientations, so the reconstruction algorithm must correctly determine and resolve the rotation of each piece.

1) Image Enhancement

Reliable edge detection and matching depend on well-conditioned inputs. The library must provide, from scratch:

- **Noise reduction** by means, Gaussian (kernel derived from size and standard deviation), and median filtering; any loop required by the median filter must be justified.
- **Contrast adjustment** by histogram equalization and by contrast stretching, with the underlying histogram computed by the library.
- **Sharpening** by unsharp masking or a Laplacian operator built on the convolution routine.
- **Thresholding** by global, Otsu, and adaptive (mean or Gaussian) methods, used subsequently to separate pieces from the background.

2) Edge Detection

The library must implement the Sobel and Prewitt gradient operators, returning both magnitude and orientation, and a complete Canny detector comprising Gaussian smoothing, gradient computation, non-maximum suppression, double thresholding, and hysteresis-based edge linking. Parameter choices must be stated and their effect illustrated on representative pieces.

3) Piece Segmentation and Contour Extraction

From the enhanced image the library must isolate the individual pieces and recover their boundaries:

- Produce a foreground mask separating pieces from the background using the thresholding routines above.
- Label connected components, from scratch, to assign a distinct identity to each piece.
- Trace the boundary of each piece
- Extract each piece to its bounding box and store its mask, contour, and a normalized orientation.

4) Piece-Edge Description

Each of the piece's four sides must be characterized so that candidate neighbors can be compared. The library must locate the four corners on the contour and thereby divide the boundary into four sides; classify every side as a **tab** (protrusion), a **blank** (indentation), or a **flat** (a straight border edge) from its geometry; and sample a strip of colour along the interior of each side to serve as a photometric signature for matching.

5) Piece-edge matching

Two puzzle pieces either belong next to each other or they don't, and the library must compute a single number that says how good a match a pair of sides is. This number should reflect two things. First, does the shape fit “a tab (a bump) has to slot into a blank (a notch), and a flat side means the piece sits on the border or corner”. Second, does the picture line up “the colours along the two touching edges should continue smoothly from one piece into the other, which you can measure by comparing the strip of colour on each edge (for example, by adding up the squared differences between them)”. The report must state the exact formula you use and how much weight you give to shape versus colour.

---

6) Assembly Algorithm

The compatibility measure is coupled with a greedy best-first search to reconstruct the puzzle. The algorithm initialises a placement grid from a corner or border piece and, at each iteration, selects the unused piece and orientation that minimise the total edge dissimilarity with the already-placed neighbours, proceeding until every cell is filled. The report must specify the tie-breaking rule, the handling of dead ends and unplaceable pieces, and must guarantee that the algorithm returns the best arrangement obtained, even when incomplete. Where the difficulty tier includes rotated pieces, each piece's rotation is resolved during placement.

Milestone 2

Develop two fundamentally different machine-learning models to reconstruct the same jigsaw puzzles used in Milestone 1. The selected models are a **Siamese Convolutional Neural Network** and a **Graph Neural Network**. Both models must be trained, validated, and tested using the same dataset. Each model must produce the reconstructed puzzle image together with the predicted position and orientation of every piece and a numerical measure of reconstruction quality.

---

1) Dataset Preparation

Students must prepare the data provided for both models by generating positive and negative side-pair samples. Positive samples must represent sides that are true neighbours, while negative samples must represent sides that do not belong together. Data augmentation may be applied to the training set and may include rotation, noise, illumination variation, contrast variation, and colour variation. The same dataset split must be used for both models to ensure a fair comparison.

---

2) Model Development

Develop and implement two fundamentally different machine-learning models:

- A Siamese Convolutional Neural Network.
- A Graph Neural Network.

Students are responsible for researching the architecture, input representation, training procedure, and implementation requirements of both models.

Each model must determine the compatibility between candidate puzzle-piece sides and provide:

- A prediction indicating whether two sides are neighbours.
- A numerical compatibility score.
- The predicted matching sides.
- The required relative orientation between the pieces.

Both models must use the same dataset splits and evaluation conditions to ensure a fair comparison.

The compatibility scores generated by each model must be passed to the puzzle assembly algorithm to determine the position and orientation of every piece and generate the reconstructed puzzle image.

The two models must be implemented independently. Changing only the number of layers, activation functions, optimisation parameters, or hyperparameters does not qualify as using two different models.

---

3) Model Training

Both models must be trained, validated, and tested using the same dataset splits to ensure a fair comparison.

Students must determine the most suitable data representation, target outputs, loss functions, optimisation method, and training procedure for each model. These choices must be supported by appropriate research and justified in the report.

For each model, the report must state:

- The model architecture.
- The input and output representations.
- The selected loss function.
- The optimiser and learning rate.
- The batch size.
- The number of training epochs.
- The data-augmentation methods.
- The training and validation results.
- The method used to select the final trained model.

Training performance must be monitored to identify problems such as overfitting or underfitting. The final evaluation must be performed only on the unseen testing set.

---

4) Puzzle Assembly

The compatibility scores generated by each model must be used by the same assembly algorithm to determine the position and orientation of all pieces and produce the final reconstructed puzzle. The algorithm must ensure that each piece is used once, border constraints are respected, and the arrangement maximises compatibility or minimises matching error.

---

5) Performance Evaluation

Both models must be evaluated on the same unseen testing puzzles and compared with the classical method from Milestone 1. Accuracy must be interpreted in terms of actual reconstruction performance, not merely as isolated numerical values.

The comparison must consider:

- Matching and neighbour accuracy.
- Piece position and orientation accuracy.
- Complete reconstruction accuracy.
- Reconstruction-quality score.
- Execution and training time.
- Model size and memory requirements.

The report must identify the most accurate and computationally efficient method and discuss how performance changes with puzzle size.

GitHub Repository Requirements

All source code, documentation, tests, results, and supporting files for Milestone 1 must be submitted through a GitHub repository. The repository must be accessible to the teaching team before the submission deadline and must remain accessible throughout the assessment period.

The repository must be organised using a clear and consistent folder structure. The following structure is recommended:

```
project-repository/
│
├── README.md
├── requirements.txt
├── main.py
│
├── src/
│   ├── __init__.py
│   ├── enhancement.py
│   ├── thresholding.py
│   ├── edge_detection.py
│   ├── segmentation.py
│   ├── contour_extraction.py
│   ├── piece_description.py
│   ├── edge_matching.py
│   ├── assembly.py
│   └── evaluation.py
│
├── tests/
│   ├── test_enhancement.py
│   ├── test_thresholding.py
│   ├── test_edge_detection.py
│   ├── test_segmentation.py
│   ├── test_piece_description.py
│   ├── test_edge_matching.py
│   └── test_assembly.py
│
├── data/
│   ├── input/
│   ├── ground_truth/
│   └── sample_pieces/
│
├── results/
│   ├── enhanced_images/
│   ├── masks/
│   ├── contours/
│   ├── edge_visualisations/
│   ├── reconstructed_images/
│   └── evaluation_results/
│
├── notebooks/
│   └── demonstration.ipynb
│
└── report/
    └── milestone_1_report.pdf
```